Google Pub/Sub - Cheat Sheet

Google Pub/Sub is a fully managed, global messaging service on GCP. Publishers send messages to a topic, subscribers read them from a subscription, and Google runs the brokers, storage, and scaling. Delivery is asynchronous and at-least-once by default. There are no partitions to size and no cluster to operate.

The one idea that trips up everyone coming from Kafka or RabbitMQ: the backlog lives on the subscription, not the topic. A topic is just a named routing point. Every subscription attached to it gets its own independent copy of the stream, with its own acks and its own retention.

The core model

Three pieces, and a topic can have many subscriptions.

Piece	Role
Topic	Named endpoint publishers send to
Subscription	An independent message queue fed from one topic
Publisher	Sends messages to a topic
Subscriber	Reads and acks messages from a subscription

```
                    ┌──> subscription A ──> subscribers (load balanced)
publisher ──> topic ┤
                    └──> subscription B ──> subscribers (a different service)
```

Two patterns fall out of this:

- **Fan-out**: attach several subscriptions to one topic. Each gets every message. This is how two unrelated services both react to `order.created`.
- **Load balancing**: run several subscribers on one subscription. Each message goes to exactly one of them. This is how you scale a worker pool.

Topics and subscriptions

Setting	Where	What it does
Message retention	Subscription	How long unacked messages are kept (default 7 days, up to 31)
Retain acked	Subscription	Keep acked messages too, so you can replay them
Ack deadline	Subscription	How long a subscriber has to ack before redelivery (default 10s, max 600s)
Dead-letter topic	Subscription	Where messages go after too many failed deliveries
Filter	Subscription	Attribute filter, set at creation, immutable
Schema	Topic	Optional Avro or protobuf schema enforced on publish
Ordering	Subscription	Deliver messages with the same ordering key in order

Delivery: push vs pull

A subscription is either pull or push. This is set once, per subscription.

Mode	How it works	Fits
Pull	Subscriber opens a streaming connection and receives messages as they arrive	Long-running workers, high throughput
Push	Pub/Sub POSTs each message to an HTTPS endpoint you own	Serverless (Cloud Run, Cloud Functions), no polling
BigQuery	Messages written straight to a BigQuery table, no subscriber code	Analytics ingestion
Cloud Storage	Messages batched into files in a GCS bucket	Archival, batch pipelines

Pull is the default and what the client libraries use (streaming pull, under the hood). Push is the right call when you already run an HTTP service and would rather be called than poll.

Acknowledgement

A subscriber must ack a message before its ack deadline expires. If it does not, Pub/Sub treats the message as unprocessed and redelivers it.

- Default ack deadline is 10 seconds, configurable up to 600.
- The client library extends the deadline automatically while your handler runs (lease management), so a slow handler is not cut off at 10 seconds.
- Ack removes the message from the subscription. Nack (or letting the deadline lapse) triggers redelivery, backed off by the retry policy.

Ack is best-effort by design. A successful ack that never reaches the server still leads to redelivery. That is why at-least-once is the baseline.

Delivery guarantees

- **At-least-once (default).** Every message is delivered one or more times. Duplicates happen. Make handlers idempotent.
- **Exactly-once (opt-in).** Enable it on the subscription and Pub/Sub will not redeliver a message that was successfully acked, within a region. It costs some latency.
- **Ordering (opt-in).** Set an ordering key on publish and enable message ordering on the subscription. Messages sharing a key are delivered in order, within a region. Without a key, order is not guaranteed.

Ordering and exactly-once both narrow to a single region. Publish through a regional endpoint when you use them.

Retention and replay

Every subscription keeps unacked messages for its retention window (7 days by default, up to 31). Turn on “retain acked messages” and you can rewind.

Two ways to replay:

- **Seek to a timestamp**: reset the subscription to any point inside the retention window. Every message after that time is redelivered.
- **Seek to a snapshot**: a snapshot captures the ack state of a subscription at a moment. Seeking to it restores exactly that state. Useful before a risky deploy.

```
# rewind a subscription to 1 hour ago
gcloud pubsub subscriptions seek orders-worker \
  --time=$(date -u -v-1H +%Y-%m-%dT%H:%M:%SZ)
```

Dead-letter topics and retry

When a message fails repeatedly, forward it instead of looping forever.

- Set a **dead-letter topic** and a **max delivery attempts** (5 to 100) on the subscription. After that many failed deliveries, Pub/Sub publishes the message to the dead-letter topic.
- The **retry policy** controls redelivery spacing with exponential backoff: `min-retry-delay` (default 10s) and `max-retry-delay` (default 600s).
- The Pub/Sub service account needs publisher rights on the dead-letter topic and subscriber rights on the subscription, or dead-lettering silently fails.

Flow control and filtering

- **Flow control** is client-side. Cap outstanding work with `setMaxOutstandingElementCount` and `setMaxOutstandingRequestBytes` so a burst does not overwhelm one subscriber.
- **Filtering** is server-side. A subscription filter on message attributes means Pub/Sub only delivers matching messages, and you are not billed for the rest. The filter is fixed at creation.

```
attributes.type = "order.created" AND attributes.region = "us"
```

gcloud CLI

```
# topic and subscription
gcloud pubsub topics create orders
gcloud pubsub subscriptions create orders-worker \
  --topic=orders \
  --ack-deadline=30 \
  --dead-letter-topic=orders-dlq \
  --max-delivery-attempts=5

# publish and pull
gcloud pubsub topics publish orders --message='{"id":42}' \
  --attribute=type=order.created
gcloud pubsub subscriptions pull orders-worker --auto-ack --limit=10
```

Java client

The client is `com.google.cloud:google-cloud-pubsub`.

Publish

```
TopicName topicName = TopicName.of(projectId, "orders");
Publisher publisher = Publisher.newBuilder(topicName).build();
try {
    PubsubMessage message = PubsubMessage.newBuilder()
            .setData(ByteString.copyFromUtf8("{\"id\":42}"))
            .putAttributes("type", "order.created")
            .build();

    ApiFuture<String> future = publisher.publish(message);
    String messageId = future.get();   // blocks; usually chained async
} finally {
    publisher.shutdown();
    publisher.awaitTermination(1, TimeUnit.MINUTES);
}
```

Subscribe (streaming pull)

```
ProjectSubscriptionName subscription =
        ProjectSubscriptionName.of(projectId, "orders-worker");

MessageReceiver receiver = (message, consumer) -> {
    try {
        process(message.getData().toStringUtf8());
        consumer.ack();
    } catch (Exception e) {
        consumer.nack();   // redeliver, backed off by the retry policy
    }
};

Subscriber subscriber = Subscriber.newBuilder(subscription, receiver)
        .setFlowControlSettings(FlowControlSettings.newBuilder()
                .setMaxOutstandingElementCount(1000L)
                .build())
        .build();

subscriber.startAsync().awaitRunning();
```

The publisher batches messages by size and time under the hood, and the subscriber runs the receiver on a thread pool. Both are meant to be long-lived. Build one, reuse it.

Pub/Sub vs Kafka vs RabbitMQ

Aspect	Pub/Sub	Kafka	RabbitMQ
Operation	Fully managed, serverless	Self or vendor managed cluster	Self or vendor managed broker
Scaling	Automatic, global	Manual, partition count	Manual, node and queue tuning
Backlog lives on	Subscription	Partition (shared log)	Queue
Ordering	Per ordering key, opt-in	Per partition	Per queue
Replay	Seek to time or snapshot	By offset	No
Routing	Topic plus attribute filter	Topic and partition	Rich exchanges and bindings
Delivery	At-least-once, exactly-once opt-in	At-least-once, exactly-once with config	At-least-once
Best for	GCP-native async work, autoscaling	High-volume event logs	Complex routing, task queues

Pick Pub/Sub when you are on GCP and want messaging without running a cluster. It trades Kafka's raw throughput ceiling and RabbitMQ's routing richness for zero operations and global reach.

Auth and IAM

Access is controlled by IAM, not broker credentials. The roles that matter:

- `roles/pubsub.publisher` to publish to a topic.
- `roles/pubsub.subscriber` to pull from a subscription.
- Grant them to service accounts, scoped per topic or subscription, not project-wide.

Best practices

- **Make handlers idempotent.** At-least-once means you will process some messages twice. Dedupe on a message ID or business key.
- **Reuse the Publisher and Subscriber.** They batch and pool internally. Creating one per message throws that away.
- **Set flow control on subscribers.** Without it, a backlog spike floods one worker's memory.

- **Wire a dead-letter topic.** It is the only clean exit for a message that keeps failing.
- **Only turn on ordering when you need it.** It caps throughput and pins you to a region.
- **Filter at the subscription.** Server-side filtering cuts delivery and cost, and the filter is free.
- **Ack after the work is durable, not before.** Ack means “I am done”, so ack only once the result is safely written.
- **Grant IAM narrowly.** Per-resource roles on service accounts, never broad project roles.

Common gotchas

- **The topic holds nothing.** Publish to a topic with no subscription and the message is gone. Create the subscription first.
- **Duplicates are normal.** At-least-once is the default. Handlers that assume exactly-once will double-process on redelivery.
- **Ack deadline is not your processing budget.** The default is 10 seconds, but the client library extends it while you work. Do not conflate the two.
- **Ordering needs a key and an enabled subscription.** Setting an ordering key alone does nothing if the subscription has ordering off.
- **Exactly-once is regional.** It does not hold across regions, and it adds latency.
- **Dead-lettering fails silently without IAM.** The Pub/Sub service account must have rights on the dead-letter topic, or messages just keep retrying.
- **Filters are immutable.** You cannot change a subscription’s filter later. Recreate the subscription to change it.
- **Seek only reaches inside retention.** You cannot replay past the retention window, and acked messages are gone unless you retained them.
- **Push endpoints must return 2xx fast.** A slow or failing endpoint looks like a nack and triggers redelivery and backoff.
- **Pub/Sub Lite is retired.** The cheaper zonal variant has been shut down. Use standard Pub/Sub.

Quick reference

Concept	Key fact
Topic	Named routing point, stores no backlog itself
Subscription	Independent queue per consumer, holds the backlog
Fan-out	Many subscriptions on one topic, each gets all messages
Load balancing	Many subscribers on one subscription, one gets each message
Pull	Subscriber receives over a streaming connection (default)
Push	Pub/Sub POSTs to your HTTPS endpoint
Ack deadline	Default 10s, max 600s, auto-extended by the client
Delivery	At-least-once default, exactly-once opt-in
Ordering	Per ordering key, opt-in, regional
Retention	Unacked kept 7 days by default, up to 31
Replay	Seek to a timestamp or a snapshot
Dead-letter topic	Target after max delivery attempts (5 to 100)
Retry policy	Exponential backoff, min 10s, max 600s
Flow control	Client-side cap on outstanding messages and bytes
Filter	Server-side attribute match, set at creation
Auth	IAM roles: `pubsub.publisher`, `pubsub.subscriber`
Java client	`com.google.cloud:google-cloud-pubsub`